

Grocery Delivery System Specifications

Technical Architecture, Database Schema & Integration Guide

Frontend: Flutter (Android Mobile App)

Backend: NestJS (REST API & WebSockets)

Payments: M-Pesa, Mixx by Yas, Airtel Money

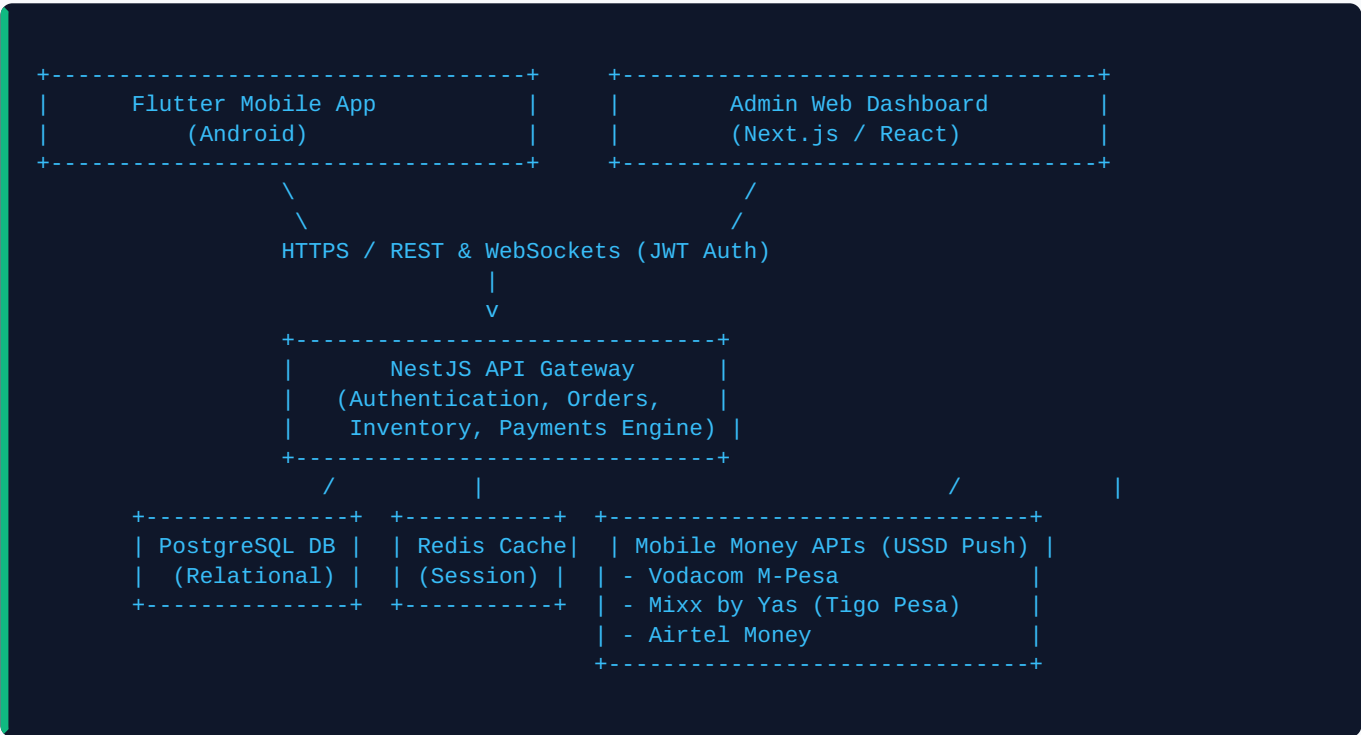
Admin Web: Next.js / React Dashboard

Database: PostgreSQL (Relational DB)

Document Version: 1.0.0

1. System Architecture Overview

The platform consists of a cross-platform mobile client built with **Flutter**, a management dashboard built with **Next.js**, and a centralized **NestJS** backend engine connected to a high-performance **PostgreSQL** database.



2. PostgreSQL Database Schema Definition

```
-- Enums Definition
CREATE TYPE user_role AS ENUM ('CUSTOMER', 'ADMIN', 'DELIVERY_AGENT');
CREATE TYPE order_status AS ENUM ('PENDING', 'PAID', 'PROCESSING', 'DISPATCHED', 'DELIVERED', 'CANCELLED');
CREATE TYPE payment_provider AS ENUM ('MPESA', 'MIXX_BY_YAS', 'AIRTEL_MONEY');
CREATE TYPE payment_status AS ENUM ('INITIATED', 'PENDING', 'SUCCESSFUL', 'FAILED');

-- Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name VARCHAR(120) NOT NULL,
  phone_number VARCHAR(15) UNIQUE NOT NULL,
  email VARCHAR(150) UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
```

```

        role user_role DEFAULT 'CUSTOMER',
        is_active BOOLEAN DEFAULT TRUE,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

-- Categories
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    icon_url TEXT,
    is_active BOOLEAN DEFAULT TRUE
);

-- Products Table
CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id INT REFERENCES categories(id) ON DELETE SET NULL,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
    unit VARCHAR(30) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
    image_url TEXT,
    is_available BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Orders Table
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    total_amount NUMERIC(12, 2) NOT NULL,
    delivery_fee NUMERIC(10, 2) DEFAULT 0.00,
    status order_status DEFAULT 'PENDING',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Order Items
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id) ON DELETE CASCADE,
    product_id UUID REFERENCES products(id),
    unit_price NUMERIC(12, 2) NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    subtotal NUMERIC(12, 2) NOT NULL
);

-- Payment Transactions
CREATE TABLE payment_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id) ON DELETE RESTRICT,
    provider payment_provider NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    amount NUMERIC(12, 2) NOT NULL,
    reference_id VARCHAR(100) UNIQUE,
    checkout_request_id VARCHAR(150) UNIQUE,
    status payment_status DEFAULT 'INITIATED',
    response_payload JSONB,

```

```

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Query Performance Indexes
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_payments_order ON payment_transactions(order_id);
CREATE INDEX idx_payments_reference ON payment_transactions(reference_id);

```

3. Component Specifications

A. Customer Flutter Application (Android)

- **Product Catalog:** Real-time product search, filtering, and stock availability verification.
- **Cart Management:** State management implemented via `flutter_bloc` or `Riverpod` with offline persistence using `Hive`.
- **Mobile Payments:** Support for USSD Push triggering across Vodacom M-Pesa, Mixx by Yas (Tigo Pesa), and Airtel Money with real-time status polling.

B. Backend Engine (NestJS)

- **Auth Module:** Phone-based OTP / JWT authentication with role-based access control.
- **Inventory & Orders:** Automated lock on product stock during checkout to prevent double-allocation.
- **Payment Engine:** Handles direct REST callbacks from mobile money operators and executes webhook signature verifications.

C. Admin Web Dashboard (Next.js)

- **Inventory Control:** Stock entry, price updates, category creation, and low-stock alerts.
- **Order Fulfillment:** Real-time tracking of order lifecycle (Pending → Processing → Dispatched → Delivered).
- **Financial Auditing:** Analytics and transaction breakdowns grouped by payment provider.

4. API Endpoint Specifications

Method	Endpoint	Access Level	Description
POST	/api/v1/auth/register	Public	Register new customer account
POST	/api/v1/auth/login	Public	Authenticate user and return JWT access token
GET	/api/v1/products	Public	Paginated listing of products and stock status
POST	/api/v1/orders	Customer	Create a new order and lock items in stock
POST	/api/v1/payments/initiate	Customer	Trigger USSD Push prompt to mobile wallet
POST		Webhook	

Method	Endpoint	Access Level	Description
	/api/v1/payments/callback/:provider		Process payment completion callback from MNO
PATCH	/api/v1/orders/:id/status	Admin	Update order status in fulfillment pipeline